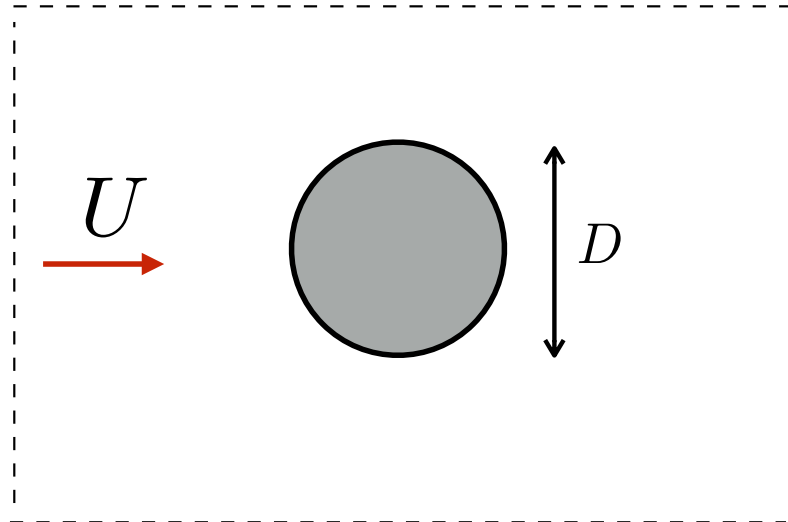


The “flow past a circular cylinder”



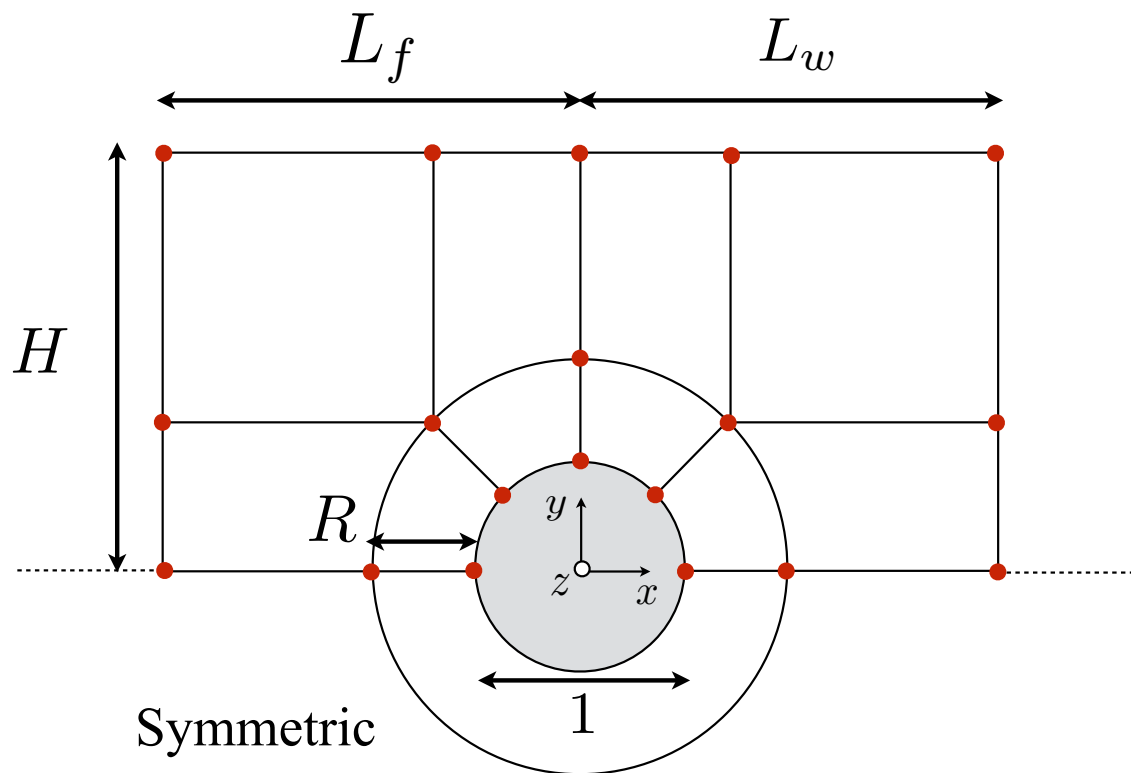
- A canonical, **two-dimensional**, **steady (or unsteady)** **incompressible** flow
- In principle, fully characterized by the Reynolds number

$$\text{Re} = UD/\nu$$

Things to note

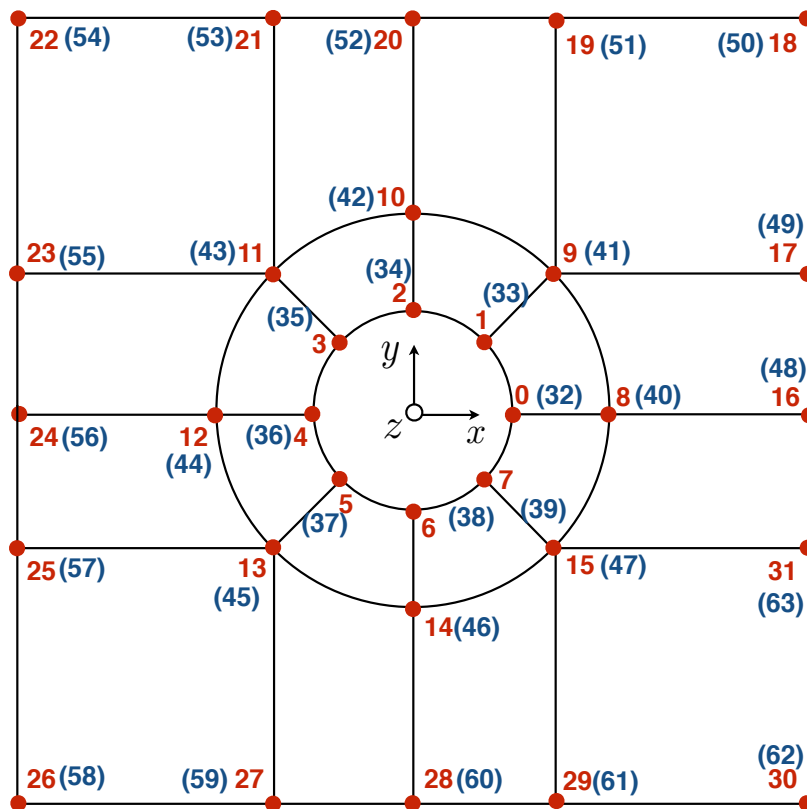
- We may want to study the flow around an *unconfined* circular cylinder, but the computational domain has finite size (by the way... also a wind- or water-tunnel has finite size...)
 - Boundary conditions *in the far field* for these flows are important
 - Size of the domain is important also. The larger, the more the velocity field will conform to an *unconfined cylinder*, but “how large is large enough?”
- Need to account for steadiness vs. unsteadiness of the flow when simulating the flow.
- Space- and time- resolution requirements for (a) boundary layer on cylinder; (b) separation point/region; (c) time accuracy if interested in vortex shedding

Mesh (geometry)



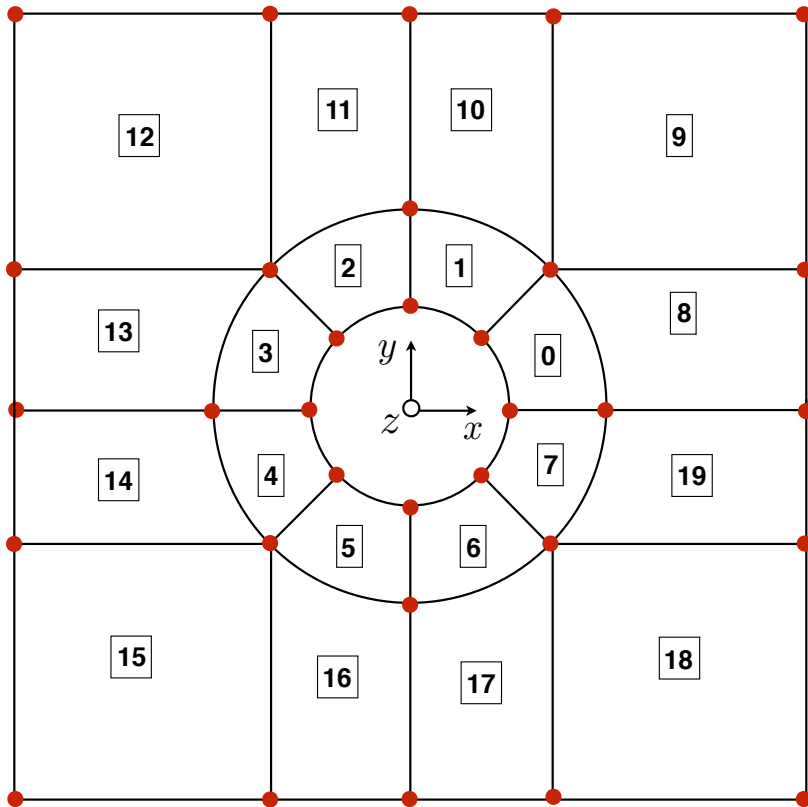
- Note the orientation of the axis according to the right-hand-rule
- The diameter of the cylinder is 1
- Define other mesh-related sizes as variables to ensure flexibility in the mesh, **should you need to change it**
- **Tip.** Use MATLAB to generate the blockMeshDict for added flexibility

Mesh (vertices)



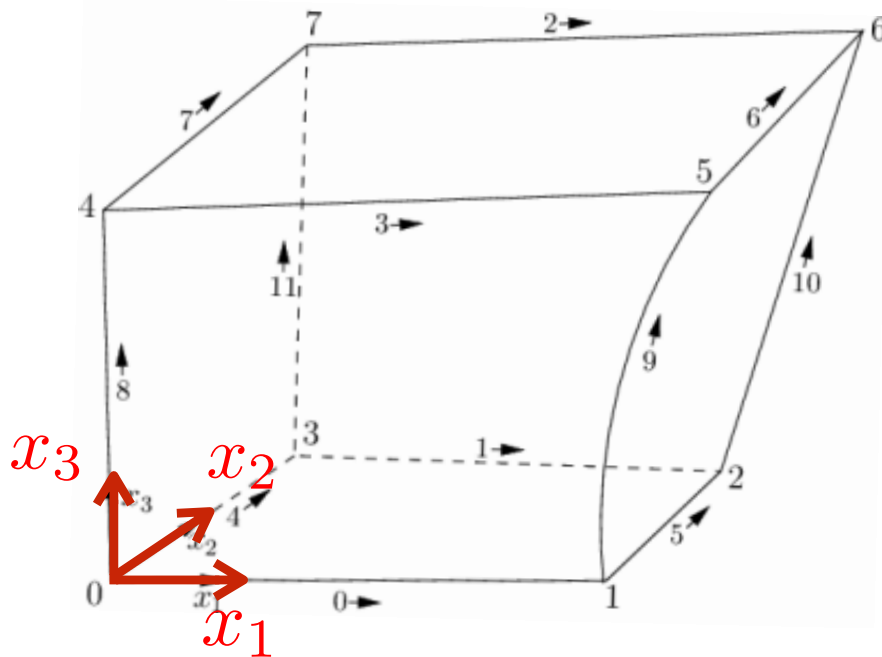
- Assign a number to each vertex, starting from 0 (blockMesh requires it)
- Choose a geometrical ordering that makes sense to you and stick to it
- Work with one x-y plane first, then duplicate the points
- Here we have $32 + 32 = 64$ points, with indices $0, \dots, 63$

Mesh (blocks)



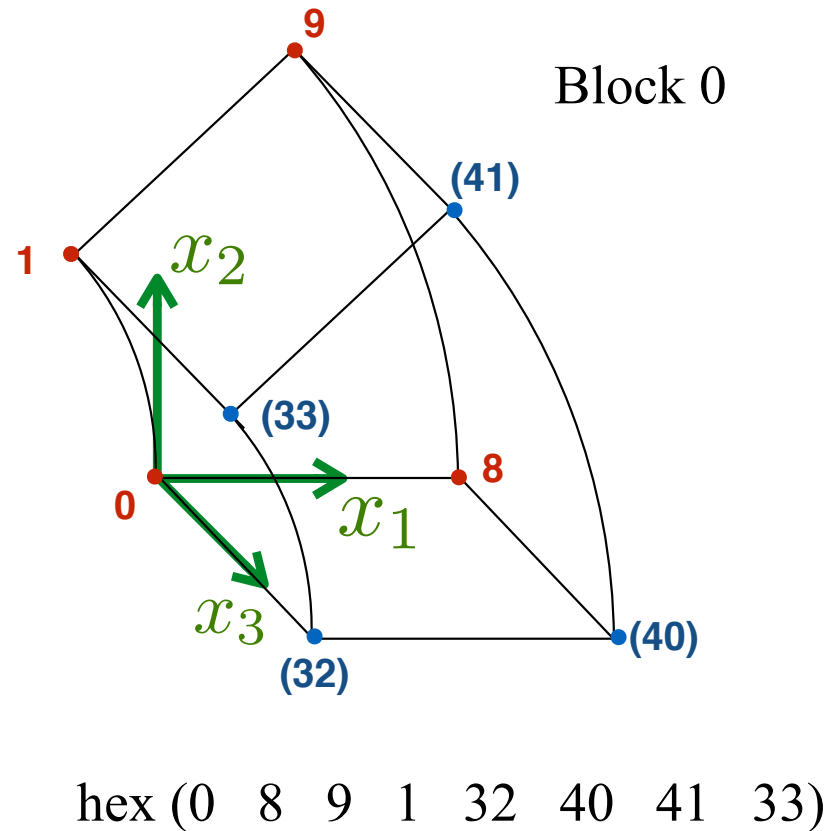
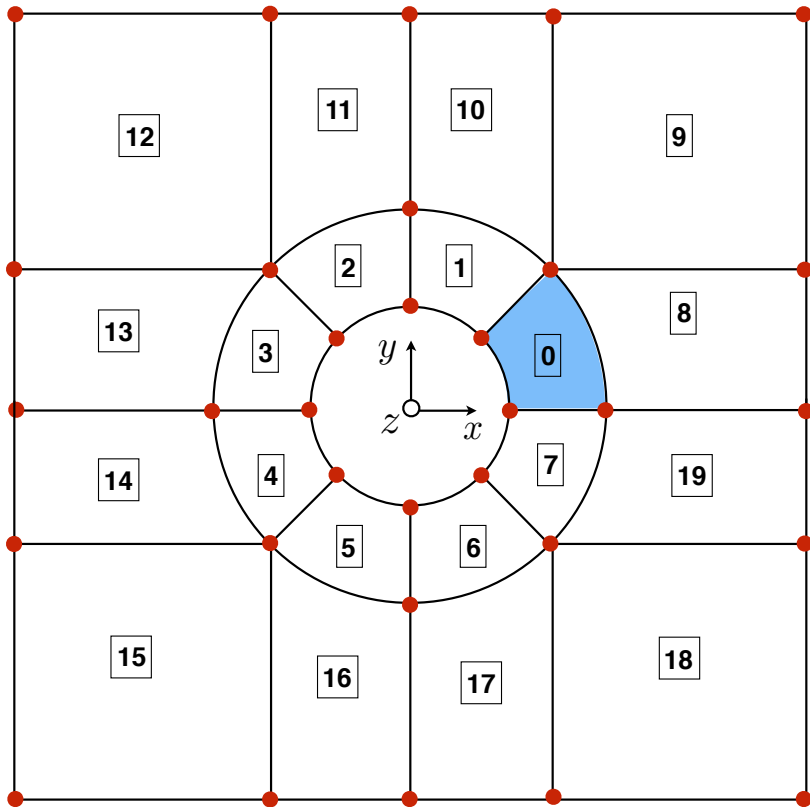
- A block consists of 8 vertices.
- Assign a number to each block, starting from 0
- Choose a geometrical ordering that makes sense to you and stick to it
- Here we have 20 blocks, numbered 0,...,19

Blocks (local coordinates)

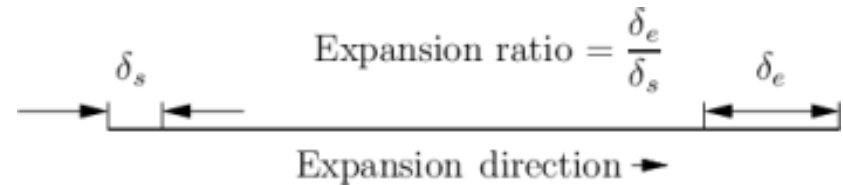
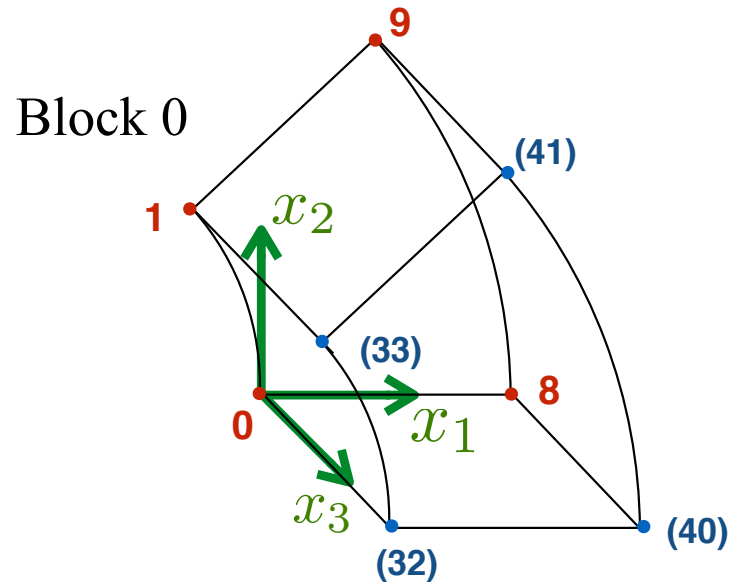


- Follow the right-hand-rule
- Orient x_1 and x_2 adequately and make a note of their orientation if using grading
- Orient x_3 consistently with the vertices and the sign of their component x_3

Blocks (local coordinates)



Blocks (local coordinates)



```
hex (0 8 9 1 32 40 41 33) // vertex list
( 20 45 1 ) // nr cells in each (local) direction (nx1, nx2, nx3)
simpleGrading (2.0 1.0 1.0) // expansion ratio in each (local) direction (ex1, ex2, ex3)
```

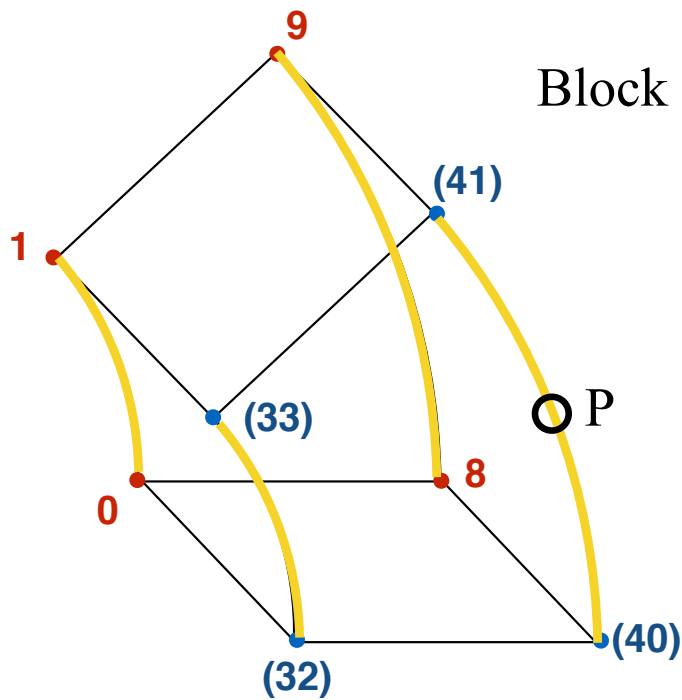

Why use grading?

- Mostly, to use more or less resolution where needed.
- Examples
 - Boundary layers require the highest resolution
 - “Far field” does not require much resolution (in general).
- More grid points (or cells) imply greater computational time! So, saving points means saving resources (computing and processing time, your salary/time, electricity, and up-front costs for hardware, storage space, etc.)

Edges

- If nothing is specified, blockMesh takes them as segments connecting each pair of vertices in each block: 12 edges per block, shared among blocks.
- You have the flexibility to specify the “shape” of an edge rather freely. In this assignment we use the “arc” shape, but many more are available (even “splines”, whereby you just specify a set of points to fit a spline).

Edges



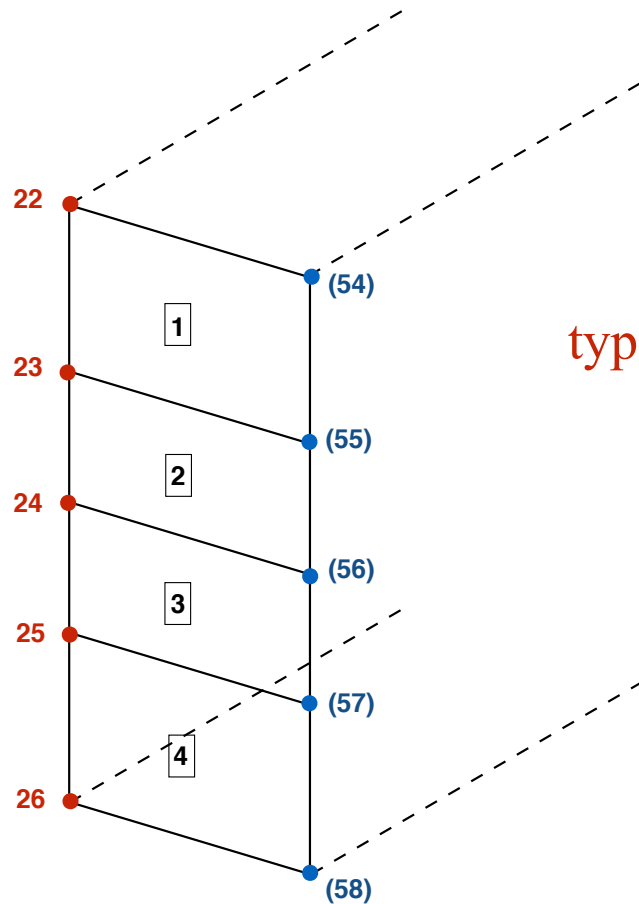
- You will need to specify 32 arcs, four per block with curved boundaries
- Block 0 has 4 arcs
- Each arc requires two vertices and the (x,y,z) coordinates of an extra point through which the arc goes through

arc 40 41 (xP yP zP)

Faces

- A face is specified as a list of 4 vertices on a plane.
- We need to worry about “external” faces only, i.e. those that do not simply separate two blocks, rather are the “boundary” of the domain.
- Faces need to be identified and grouped, so that boundary conditions may be assigned to each “group”.
- Here, you will need to define 5 distinct groups of faces: inlet, outlet, top, bottom, cylinder
- If an external face is not listed in a user-defined group, it is considered part of the *empty* group.

The inlet patch



type of faces in
group

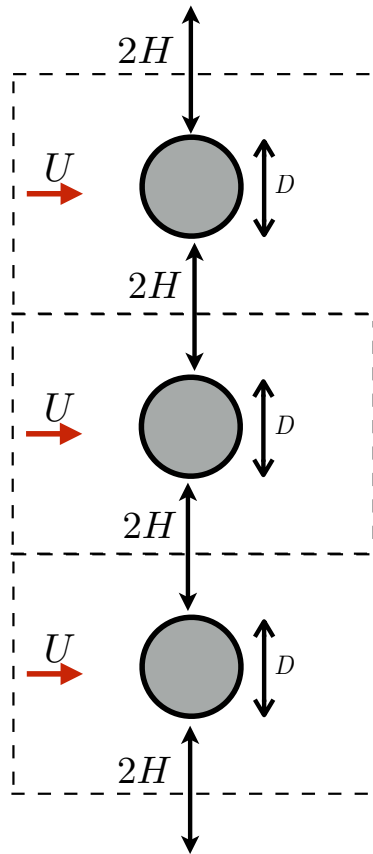
```
inlet ← group name (your choice,  
{                                     but make a note of it)  
  type patch;  
  faces  
  (  
    (22 54 55 23) ← list of faces, each  
    (23 55 56 24)   with 4 vertices,  
    (24 56 57 25)   that belong to  
    (25 57 58 26)   the group  
  );  
}
```

- Again, decide an ordering and stick to it in order to avoid mistakes

Faces types

- In this assignment, define:
 - Top and bottom as type “**symmetryPlane**”
 - Cylinder as type “**wall**”
 - Inlet and outlet as type “**patch**”

What does a symmetry plane do?



- While you are simulating only one cylinder, a **symmetrPlane** makes it such that you are simulating an infinite number of cylinders, stacked on top of each other.
- You need the separation between the cylinders to be large enough so that the flow field around each cylinder is independent of the separation distance

Additional advice

- Start early with generating a mesh and consider writing a MATLAB (or Python, or...) script to make the generation of blockMeshDict automatic.
- *Somewhat like building a small ship model in a bottle:* patience, patience, patience and attention to details. It really pays off to draft a drawing carefully and plan ahead.
- When you run blockMesh, take a good look at the output to screen. Take care of resolving any “warning” or “error” statements. Address each “warning” or “error” in the order they appear.
- After running blockMesh, run checkMesh to confirm the mesh is sensible. Again, take a close look at the output to screen and address issues, one by one.